WHITEWISE INTERIOR FURNITURE

DEVELOPER DOCUMENTATION

Interior Furniture Management System

Technical reference for developers and maintainers

Document version	1.0 — Working draft
Prepared for	Whitewise Interior Furniture
Prepared	October 2026

This document describes the system using the screens, routes, and implementation details shared during development. Names and behavior of individual screens may vary with the deployed build.

Contents

- 1. System overview and architecture
- 2. Technology stack
- 3. Frontend structure and routes
- 4. Administrator dashboard implementation
- 5. Backend API integration
- 6. Authentication and authorization
- 7. Local setup and execution
- 8. Testing and troubleshooting
- 9. Security, deployment, and maintenance
- 10. Verification items

1. System overview and architecture

The system is a React single-page application for furniture/interior-related operations. The frontend uses React Router for page navigation and calls a Spring Boot backend through HTTP APIs. The known administrator dashboard communicates with the backend at http://localhost:9193 and uses bearer-token authentication. Database details have appeared in earlier project work, but the exact schema for the current deployed build should be confirmed from the active backend configuration and migrations.

Logical architecture

Layer	Responsibility	Known implementation
Presentation	Renders screens, forms, navigation, and feedback.	React, JSX, Bootstrap.
Routing	Maps browser paths to React page components.	react-router-dom.
HTTP client	Sends requests to the backend and includes authentication.	Axios.
Backend API	Handles requests, validation, business rules, and authorization.	Spring Boot; exact modules should be verified.
Persistence	Stores application records.	MySQL has been used in project context; confirm active database and schema.

2. Technology stack

- **Frontend:** React, Vite in the local development setup, JSX, Bootstrap CSS/JavaScript, React Router, and Axios.
- **Backend:** Spring Boot REST API. The administrator dashboard's current API base URL is configured as http://localhost:9193.
- **Authentication:** bearer token retrieved from localStorage under the key token in the provided dashboard code.
- **Database:** MySQL has been used in the wider project context; confirm the current application's datasource and database name before deployment.
- **Icons:** the generated dashboard uses Bootstrap Icons class names and requires the bootstrap-icons package/import if not already installed.

3. Frontend structure and routes

The following structure reflects the component paths shared in the project. Verify exact file names and capitalization against the repository.

```
src/
App.jsx
components/
admin/
AdminDashboard.jsx
UserDashboard.jsx
login.jsx
auth/
LoginPage.jsx
SignupPage.jsx
WelcomeDashboard.jsx
home/
Home.jsx
AboutUs.jsx
layout/
Navbar.jsx
Footer.jsx
room/
AddRoom.jsx
ExistingRooms.jsx
EditRoom.jsx
RoomListing.jsx
Dahboard.jsx
service/
CustomerServices.jsx
Sales.jsx
Orders.jsx
Products.jsx
Inventory.jsx
FinishedProduct.jsx
ProductCards.jsx
CatalogPage.jsx
BillingForm.jsx
PaymentPage.jsx
```

Known route map

Path	Component
/	Home
/existing-rooms	ExistingRooms
/add-room	AddRoom
/edit-room/:id	EditRoom
/browse-all-rooms	RoomListing
/dashboard	Dashboard / Dahboard.jsx
/customer-services	CustomerServices
/sales	Sales
/orders	Orders
/products	Products
/inventory	Inventory
/finishedproduct	FinishedProduct
/cardproduct	ProductCards
/catalog	CatalogPage
/login	admin/login
/loginpage	LoginPage
/signup	SignupPage
/dashboard2	WelcomeDashboard
/admin/dashboard	AdminDashboard
/user/dashboard	UserDashboard
/billing	BillingForm
/payment	PaymentPage
/about-us	AboutUs

Naming caution: the provided import is ./components/room/Dahboard (spelled Dahboard). Do not rename it without updating the import and file name together.

4. Administrator dashboard implementation

The administrator page combines API-backed account information with local UI state.

State

- users: list returned from GET /admin/users.
- currentAdmin: account data returned from GET /admin/users/me.
- loginHistory: data returned from GET /admin/users/me/history.
- form: selected user details for editing.
- addUserForm: fields for creating a user.

- reset: selected user ID and new password for password reset.
- showUsers, showAddUser, and showLoginHistory: visibility state for collapsible UI.

Navigation buttons

The seven management buttons use React Router's useNavigate() and navigate to the existing paths: /customer-services, /sales, /orders, /products, /inventory, /dashboard, and /finishedproduct. They do not import or render the destination page directly; App.jsx resolves the route.

Current API calls in AdminDashboard.jsx

Method	Path	Purpose in UI
GET	/admin/users/me	Load signed-in administrator information.
GET	/admin/users/me/history	Load login history.
GET	/admin/users	Load user list.
PUT	/admin/users/{id}	Update a user.
DELETE	/admin/users/{id}	Delete a user.
PUT	/admin/users/{id}/password	Reset a user's password.
POST	/auth/signup	Create a user from the Add User form.
POST	/admin/users/me/profile-picture	Upload profile image using multipart form field 'file'.

These paths are read from the supplied frontend code. Confirm that the backend controllers expose the same methods and paths and that the expected request/response shapes match.

5. Backend API integration

The dashboard creates an Axios instance with baseURL set to http://localhost:9193 and an Authorization header of the form Bearer <token>. This value is read when the module initializes. If the token changes after the module is loaded, the configured header may not update automatically; consider an Axios request interceptor that reads localStorage for each request.

Suggested backend checks

- Verify controller mappings for every endpoint in the API table.
- Validate required fields and return useful HTTP status codes and error bodies.
- Ensure user IDs are validated and the caller has permission to edit, delete, or reset that account.
- Confirm multipart upload configuration, file size limits, accepted image types, and storage strategy.
- Avoid returning password hashes, secrets, tokens, or unnecessary personal data in API responses.

6. Authentication and authorization

- The UI's bearer token is read from localStorage key token. The login flow must store the token under that same key or the API calls may return 401.
- A 401 response typically means credentials are missing, invalid, or expired. A 403 typically indicates that the caller is authenticated but lacks permission.
- Frontend route visibility is not a security boundary. Enforce authorization in Spring Security/backend service methods for every protected endpoint.
- Restrict administrator-only operations server-side. Never rely solely on hiding a button in React.

- For production, use HTTPS and review whether localStorage token storage is appropriate for the application's threat model.

7. Local setup and execution

Frontend

- Install a supported Node.js version and npm.
- Open a terminal in the frontend project directory containing package.json.

```
npm install
npm run dev
```

- Open the Vite URL printed in the terminal, commonly http://localhost:5173 in the local setup.

Backend

- Open the Spring Boot backend project in the IDE or terminal.
- Configure the datasource, database credentials, API port, and JWT secret using local environment/configuration files.
- Start the backend using the project's Maven wrapper if available, or use the IDE's Spring Boot run configuration.

```
mvn spring-boot:run
```

Do not copy real production passwords, database credentials, or JWT signing secrets into source control. The command and ports may differ depending on the active project configuration.

8. Testing and troubleshooting

Test area	Minimum verification
Build	Run the frontend build and backend tests/build; resolve compile and lint errors.
Navigation	Open every route directly and through its navigation button.
Authentication	Test valid, missing, expired, and unauthorized tokens.
Administrator profile	Verify profile details and login history load for an authorized admin.
User management	Test add, list, edit, delete, and password reset with authorized accounts; test validation failures.
Profile image	Test valid image, unsupported file, oversized file, and failed network request.
Responsive layout	Check dashboard cards and the user table on desktop and mobile-sized viewports.
Payment	Test only in a provider sandbox or approved test environment if payment integration exists.

Useful diagnostics

- Browser DevTools → Console: identify React errors and failed imports.
- Browser DevTools → Network: inspect request URL, status code, request payload, and response body. Never share tokens or sensitive payloads.
- Backend logs: check endpoint mapping, authorization failures, validation errors, database connectivity, and stack traces.
- CORS: if the browser blocks API calls, configure Spring Boot CORS/security to allow the exact frontend origin used in development.

9. Security, deployment, and maintenance

- Use separate development and production configuration; keep secrets outside committed source files.

- Enable HTTPS for hosted environments and restrict CORS to trusted origins.
- Use database backups and test restoration procedures before production changes.
- Validate and sanitize user input on the backend; apply least-privilege database and application permissions.
- Use authorization checks on all sensitive API routes, including profile changes, account management, and payment-related operations.
- Set upload limits and validate image content on the server rather than trusting file extensions alone.
- Log security-relevant events without recording passwords, bearer tokens, or unnecessary sensitive data.
- Keep dependencies patched and run automated tests after route, API, or database changes.

10. Verification items before treating this as final production documentation

- Confirm the current repository name and production application URL.
- Confirm the active backend port, database engine/name, schema, and configuration file.
- Confirm all endpoint mappings, DTO fields, role names, and actual authorization rules.
- Confirm exact behavior of each page, especially Billing, Payment, Sales, Orders, and Finished Products.
- Confirm deployment instructions, backup/restore process, and designated support contact.

This document is a developer-oriented reference based on the routes and code shared in the conversation. It is not a substitute for reviewing the full repository, backend controllers, security configuration, database schema, and payment-provider implementation.

End of Developer Documentation